

AI Entrepreneur Coach

PATH B — GREEN RETROFIT AUDIT

Defending the assessment and proposing GSF-aligned improvements

Why this matters here

- **Artifact:** AI Entrepreneur Coach, an LLM pipeline that ranks business ideas and writes a 90-day roadmap for each user.
- **Every session costs real calls:** OpenAI, Cohere, and Pinecone calls run per session, cost and latency scale directly with usage.
- **Primary audience:** an engineering lead deciding whether to take this from prototype to production.
- **Why green here:** for a project this size, green software is a cost and latency story first, carbon is the downstream effect.

What “R” means here

R = one finished coaching report: one user's uploaded profile, TIPI answers, and constraints, ranked against the idea corpus, ending in a written report and a 90-day roadmap for their picked idea.

Where the compute actually goes

- **Ranking step, once/session:** 1 OpenAI call + up to 7 Cohere embedding calls (30-candidate batch is efficient, a 5-call matched-skills loop is not) + 1-2 Pinecone queries.
- **Low confidence retry:** would double Cohere/Pinecone calls, but repeated manual testing on a very sparse CV never triggered it.
- **Report + roadmap generation:** 1 combined OpenAI call, already efficient. Bigger model kept on purpose, already tested against a smaller one.
- **Feedback regeneration:** resends previous report + feedback history each round, on purpose, capped at 3 rounds / 300 chars per round.
- **One-time dataset build:** close to 500 OpenAI calls, developer-only, separate from the cheap seed script.
- **Pinecone index:** explicitly hosted in AWS us-east-1, stays online between sessions.

What's still a guess

- No logging or analytics yet, so token and session counts above are estimates from reading the pipeline.
- No explicit region set for OpenAI/Cohere calls, only Pinecone is explicit (us-east-1).
- Unknown how often the same idea repeats across sessions, this decides how much skill-embedding caching would save.
- Low confidence retry's real firing rate is backed by repeated manual testing, not logged data across real usage.

“Why should I believe the report/roadmap step is the biggest cost, not ranking?”

Ranking already batches well except one 5-call loop; report/roadmap uses a bigger, already-tested model and resends growing context on feedback rounds — likely more tokens overall, but unconfirmed without real logging.

Hotspots across the four pillars

Hotspot	Carbon	Energy	Hardware	Measurement
Report/roadmap generation (combined call)		Yes	Yes	Yes
Feedback regeneration (growing resend)		Yes		Yes
Repeated skill embedding per session		Yes	Yes	
Unbatched matched-skills loop (5 calls)		Yes		
Low confidence retry (ranking)		Yes	Yes	Yes
Pinecone standing index (us-east-1)	Yes		Yes	
One time dataset build (developer only)		Yes	Yes	

SOLUTION 1

Confirm feedback-regen cost stays small

PROBLEM

Every feedback round resends the full previous report JSON plus the feedback history so far, on purpose, so no earlier round gets silently dropped.

CHANGE

Feedback text is already capped at 300 characters for 3 rounds. Measure the resent report JSON size before touching anything — it's likely already a low-priority cost.

SOLUTION 2

Cache each idea's skill embedding

PROBLEM

compute_skill_fits and matched_skills re-embed each candidate idea's fixed skills_needed text with Cohere on every single session, even though that text never changes.

CHANGE

Embed each of the 463 ideas' skills once, cache by idea id. Only the user's own skills get embedded fresh. The 90-day roadmap stays generated per user, untouched.

SOLUTION 3

Batch the matched-skills check

PROBLEM

matched_skills runs once per top-5 idea in a loop, 5 separate Cohere calls, each re-embedding the same user skills again.

CHANGE

Batch into 1 call instead of 5, same threshold logic, same output, no quality change, only removes duplicate calls.

SOLUTION 4

Confirm the low-confidence retry rarely fires

PROBLEM

The retry safeguard would roughly double Cohere/Pinecone calls for sparse CVs if it fires.

CHANGE

Repeated manual testing never triggered it. Add lightweight logging to confirm at real scale, so this stays low priority based on numbers, not repeated manual tries.

SOLUTION 5 (stretch)

Guard the expensive dataset rebuild

PROBLEM

build_dataset.py costs close to 500 OpenAI calls; nothing in the code stops an accidental run instead of the cheap seed_pinecone.py.

CHANGE

Add a short “are you sure” confirmation prompt before the expensive rebuild runs.

What I'd track for 1–2 weeks

- Tokens per feedback round, split into capped feedback text vs. the resent report JSON.
- Skill embedding cache hit rate, once idea-level caching (solution 2) ships.
- Low confidence retry firing rate, and how many extra Cohere/Pinecone calls it adds.
- Cohere call count and latency for the ranking step, before/after batching matched-skills (solution 3).
- How many times `build_dataset.py` runs per month vs. `seed_pinecone.py`, after adding the confirmation guard.

Success = these numbers confirm which hotspots are real vs. already small, before committing to more redesign work.

CAVEATS

Honest limits

- All numbers above are estimates from reading the code, not measured, no logging exists yet.
- No “carbon neutral” or similar unverifiable claims are made anywhere in this audit.
- The bigger report/roadmap model and the feedback-resend design are deliberate, already-tested choices, not being second-guessed here.
- Next step: add logging, then revisit priority order using real numbers.